

12. naloga: Strojno učenje za kategorizacijo podatkov

Filip Lovšin 28242022

April 24, 2026

1 Uvod

Pri nalogi obravnavam problem ločevanja dogodkov, kjer nastane Higgsov bozon (ki je v našem primeru signal), od ostalih procesov (ki jim rečemo ozadje). Na voljo imam simuliran nabor dogodkov, kjer je vsak dogodek opisan z 18 zveznimi kinematičnimi spremenljivkami. Posamezna spremenljivka sama po sebi navadno ne loči dobro signala od ozadja, zato je smiselno uporabiti metode strojnega učenja, ki znajo iz več šibkih informacij sestaviti eno močnejšo diskriminanto.

V okviru naloge primerjam dva pristopa:

- pospešena odločitvena drevesa (BDT),
- globoke nevronske mreže (DNN).

Uspešnost metod ocenimo z ROC krivuljo in pripadajočo površino pod krivuljo (AUC). ROC prikazuje kompromis med učinkovitostjo signala (torej koliko signala obdržimo) in onesnaženostjo z ozadjem (koliko ozadja pomotoma spustimo skozi) pri različnih izbirah praga.

Glavni cilji naloge so preizkusiti tipične konfiguracije BDT in DNN ter jih primerjati z ROC/AUC, preveriti vpliv izbire vhodnih spremenljivk (npr. kaj se zgodi, če uporabimo samo nekatere) in preizkusiti vpliv hiperparametrov (npr. število dreves pri BDT, število plasti in nevronov pri DNN).

2 Teoretično ozadje

2.1 Kaj je strojno učenje

Strojno učenje (Machine Learning, ML) se danes pogosto uporablja v znanosti, posebej tam, kjer imamo veliko podatkov in želimo avtomatsko odkriti vzorce ali zgraditi klasifikator. Osnovne vrste ML običajno razdelimo na:

- nadzorovano učenje: imamo vhodne podatke in znane oznake,
- nenadzorovano učenje: oznak nimamo, algoritmi iščejo strukturo v podatkih,
- spodbujevalno učenje: učenje preko nagrad in kazni.

Pri fizikalnih analizah (in tudi pri tej nalogi) najpogosteje uporabljamo nadzorovano učenje, ker imamo simulacije, kjer je razred, torej signal znan.

Prednost ML pristopa pred ročnim rezanjem je v tem, da lahko algoritem upošteva več spremenljivk hkrati ter tudi njihove korelacije. V več dimenzijah bi bilo zelo težko ugibati optimalno analitično funkcijo ali optimalne reze, medtem ko ML model takšno funkcijo približa iz podatkov.

Recimo, da imamo učno množico

$$\mathcal{D} = \{(x_k, y_k)\}_{k=1}^N,$$

kjer je $x_k \in \mathbb{R}^M$ vektor M značilk (v našem primeru vhodnih spremenljivk), y_k pa ciljna vrednost (pri klasifikaciji tipično $y_k \in \{0, 1\}$). Želimo se naučiti model (hipotezo) $h(x)$, ki na vhodu x vrne napoved $\hat{y}$.

Učenje običajno formuliramo kot minimizacijo funkcije izgube:

$$L(h) = \frac{1}{N} \sum_{k=1}^N \ell(y_k, h(x_k)),$$

kjer ℓ meri napako napovedi na primer, pri binarni klasifikaciji pogosto uporabimo binarno navzkrižno entropijo. Po učenju model uporabimo na novih dogodkih in dobimo napovedi oziroma verjetnosti, da dogodek spada med signal.

Odločitveno drevo deluje kot zaporedje enostavnih vprašanj tipa ali je značilka večja od neke meje? Tako drevo razdeli prostor podatkov na regije, v katerih napove razred. Ker je eno drevo pogosto nestabilno in se lahko prehitro prilagodi učnim podatkom, se v praksi uporablja pospeševanje (oziroma boosting). V tem primeru zgradimo zaporedje dreves, kjer vsako novo drevo poskuša popraviti napake prejšnjih.

BDT zato predstavlja uteženo kombinacijo več dreves, kar tipično močno izboljša ločljivost signala in ozadja. V praksi uporabljamo algoritme oziroma pripomočke, kot je **CatBoost**, ki nam bo prav prišla pri tej nalogi.

Osnovni gradnik nevronske mreže je perceptron oziroma nevron, ki izračuna uteženo vsoto vhodov ter jo nelinearno preslika:

$$h_{w,b}(x) = \theta(w^T x + b),$$

kjer w predstavlja uteži, b pristranek (bias), θ pa aktivacijsko funkcijo (npr. tanh). Nevronska mreža je sestavljena iz več takšnih plasti. Globoka nevronska mreža (DNN) pomeni, da imamo več plasti in s tem večjo sposobnost modeliranja kompleksnih nelinearnih odvisnosti.

Učenje DNN spet poteka z minimizacijo izgube, le da je parametrov veliko. Zato se uporablja gradientni spust in njegove izboljšave (npr. Adam), pogosto pa še dodatne tehnike za stabilnejše učenje in manj preučenja (npr. dropout, regularizacija, batch normalizacija).

2.2 Orodja

Za implementacijo sem uporabil Pythonove knjižnice: **CatBoost** za učenje BDT modelov, **TensorFlow/Keras** za gradnjo in učenje globokih nevronske mreže in **Scikit-learn** za standardne metrike in evalvacijo (ROC, AUC, delitve množic).

3 Postopek reševanja naloge

3.1 Ideja reševanja problema

Problem je binarna klasifikacija, vsak dogodek je ali signal (Higgs) ali ozadje. Ker posamezne kinematične spremenljivke same po sebi slabo ločujejo signal od ozadja, uporabim metode strojnega učenja, ki znajo kombinirati več vhodnih spremenljivk hkrati in tako izboljšajo ločevanje.

Za primerjavo metod sem implementiral dve tipični rešitvi za tabularne podatke in sicer BDT z uporabo knjižnice **CatBoost** in DNN z uporabo **TensorFlow/Keras**.

Uspešnost ocenjujem z ROC krivuljo in AUC, kar je standardna metoda pri ločevanju signala ali ozadja.

3.2 Priprava podatkov

Podatki so shranjeni v datoteki **higgs-parsed.h5**. Iz nje preberem dve tabeli:

- **train**: uporabim jo za učenje modelov in interno validacijo med učenjem,
- **valid**: to je neodvisna validacijska množica in jo uporabim za končne AUC/ROC rezultate.

Oznaka razreda je stolpec **hlabel** (0 = ozadje, 1 = signal), ostalih 18 stolpcev pa so vhodne spremenljivke. Da ne pride do uhajanja informacij, skaliranje vedno nastavim samo na učnem delu in potem enake parametre uporabim še na validaciji.

Uporabim dve različni normalizaciji, ker se različni modeli na njih različno dobro obnašajo. Za BDT uporabim min-max preslikavo v interval [0, 1] in za DNN uporabim standardizacijo (povprečje 0, standardni odklon 1).

3.3 Model 1: BDT s CatBoost

Pri BDT uporabljam knjižnico **CatBoost**, ki implementira gradientno ojačevanje dreves. Glavna ideja je, da model zgradi zaporedje odločitvenih dreves, kjer vsako novo drevo popravlja napake prejšnjih.

V kodi nastavljam parametre števila iteracij, globine dreves in tudi učne hitrosti.

CatBoost ima vgrajeno tudi early stopping logiko (preko validacijskega seta), zato treniranje ustavim, ko se rezultat na validaciji ne izboljšuje več.

3.4 Model 2: DNN s TensorFlow/Keras

Za nevronske mreže uporabljam TensorFlow/Keras. Mreža je tipa MLP, torej zaporedje gostih plasti. Uporabim ReLu aktivacijsko funkcijo v skritih plasteh, sigmoid na izhodu ker gre za binarno klasifikacijo in binary cross entropy ko funkcijo izgube.

3.5 Merjenje uspešnosti (ROC in AUC)

Za oba modela izračunam napovedane verjetnosti signala in nato narišem ROC krivuljo. ROC krivulja pokaže razmerje med signalno učinkovitostjo in ozadno učinkovitostjo pri različnih pragih. AUC uporabim kot enostavno številčno primerjavo: večji AUC pomeni boljše ločevanje.

3.6 Študija vpliva števila spremenljivk

Ker me zanima, ali je vseh 18 spremenljivk res potrebnih, naredim tudi test top- k . Spremenljivke razvrstim na dva načina:

- po pomembnosti spremenljivk iz CatBoost,
- po mutual information z oznako razreda.

Nato za vsak k ponovno natreniram model samo na prvih k spremenljivkah in izračunam AUC. Tako dobim graf $AUC(k)$, ki pokaže, kdaj se izboljšave začnejo manjšati (saturacija) in koliko spremenljivk je približno optimalno.

3.7 Preizkus hiperparametrov

Na koncu naredim še majhen preizkus hiperparametrov:

- pri BDT spreminjam število iteracij, globino in learning rate,
- pri DNN spreminjam velikosti skritih plasti, dropout in learning rate.

Rezultate shranim v .csv datoteke, kar mi omogoča enostavno primerjanje in izbiro najboljše konfiguracije. Te datoteke tudi prilagam k poročilu v zip datoteki.

4 Rezultati, ločevanje signala od ozadja z BDT in DNN

4.1 Podatki in evalvacija

Cilj je ločiti Higgsove signalne dogodke od ozadnih dogodkov z uporabo 18 zveznih kinematičnih spremenljivk. Podatkovni vzorec je razdeljen na učno množico (uporabljena za učenje modela) in neodvisno validacijsko množico (uporabljena za končno oceno uspešnosti in ROC krivulje).

Za primerjavo različnih klasifikatorjev uporabljam ROC krivuljo in površino pod ROC krivuljo (AUC). ROC krivulja prikazuje kompromis med:

- učinkovitostjo signala (true positive rate): delež signalnih dogodkov, ki jih obdržimo,
- učinkovitostjo ozadja (false positive rate): delež ozadnih dogodkov, ki jih napačno sprejememo kot signal.

Naključni klasifikator sledi diagonalni. Boljši klasifikator ima ROC krivuljo bližje zgornjemu levemu vogalu. AUC je en sam številčni povzetek:

$$AUC = 0.5 \text{ (naključno)}, \quad AUC = 1.0 \text{ (popolno ločevanje)}.$$

4.2 Primerjava

Tabela 1 prikazuje AUC na neodvisni validacijski množici za glavne konfiguracije. Najboljši rezultat dobimo z DNN, ki uporablja batch normalizacijo in dropout.

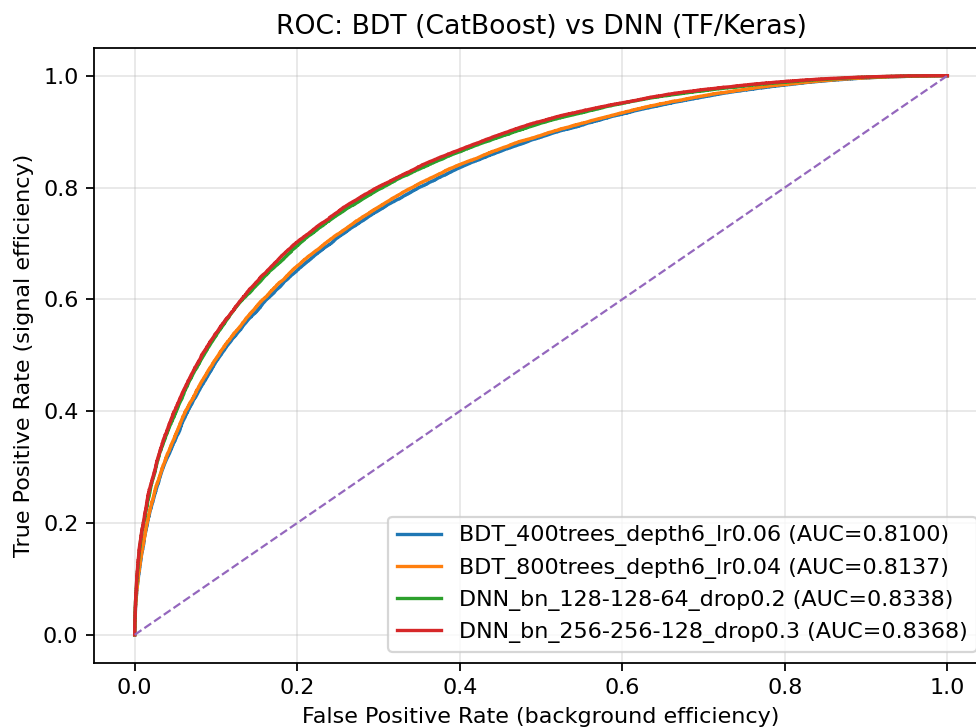


Figure 1: ROC krivulje za dve konfiguraciji CatBoost BDT in dve konfiguraciji DNN (TensorFlow/Keras). Črtnana diagonala predstavlja naključno ugibanje. Krivulji DNN sta večino časa nad krivuljama BDT, zato imata tudi višji AUC in s tem boljše ločevanje signala od ozadja.

Model/konfiguracija	AUC (valid)
DNN_bn_256-256-128_drop0.3	0.83683
DNN_bn_128-128-64_drop0.2	0.83384
BDT_800trees_depth6_lr0.04	0.81374
BDT_400trees_depth6_lr0.06	0.80998

Table 1: AUC za glavne konfiguracije, prikazane v ROC grafu.

4.3 Zgodovina učenja in preverjanje overtraininga (DNN)

Da preverim overtraining, prikažem potek učenja v odvisnosti od epohe. V grafih sta prikazana tako izguba kot AUC na učni in validacijski množici.

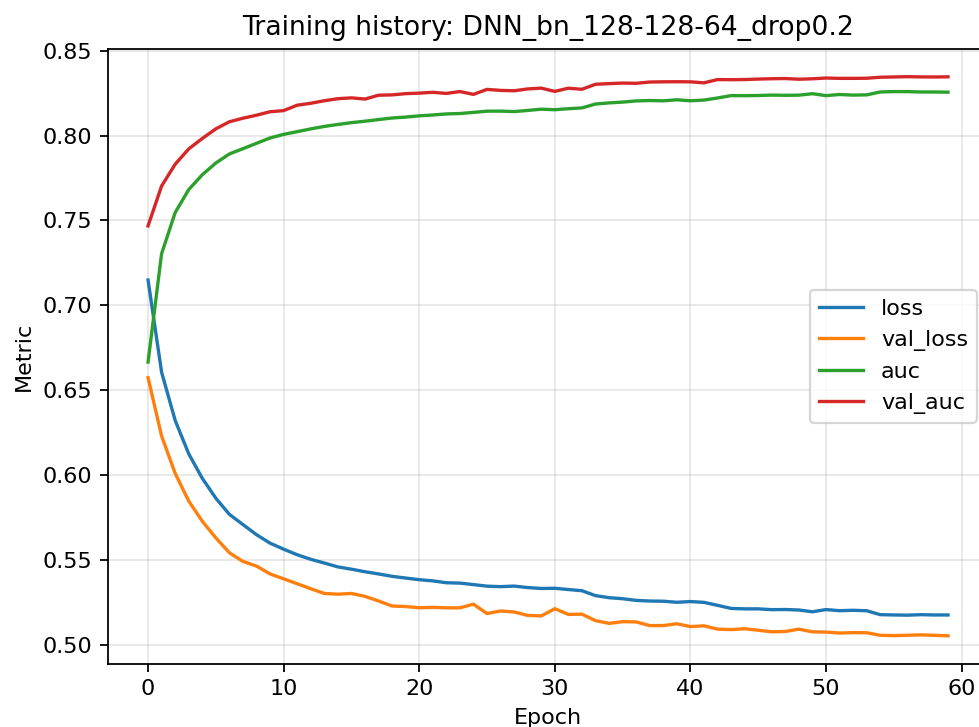


Figure 2: Zgodovina učenja za DNN z arhitekturo (128,128,64) in dropout 0.2. Validacijski AUC sledi učnemu AUC in nato saturira, kar kaže na stabilno učenje brez izrazitega preučenja.

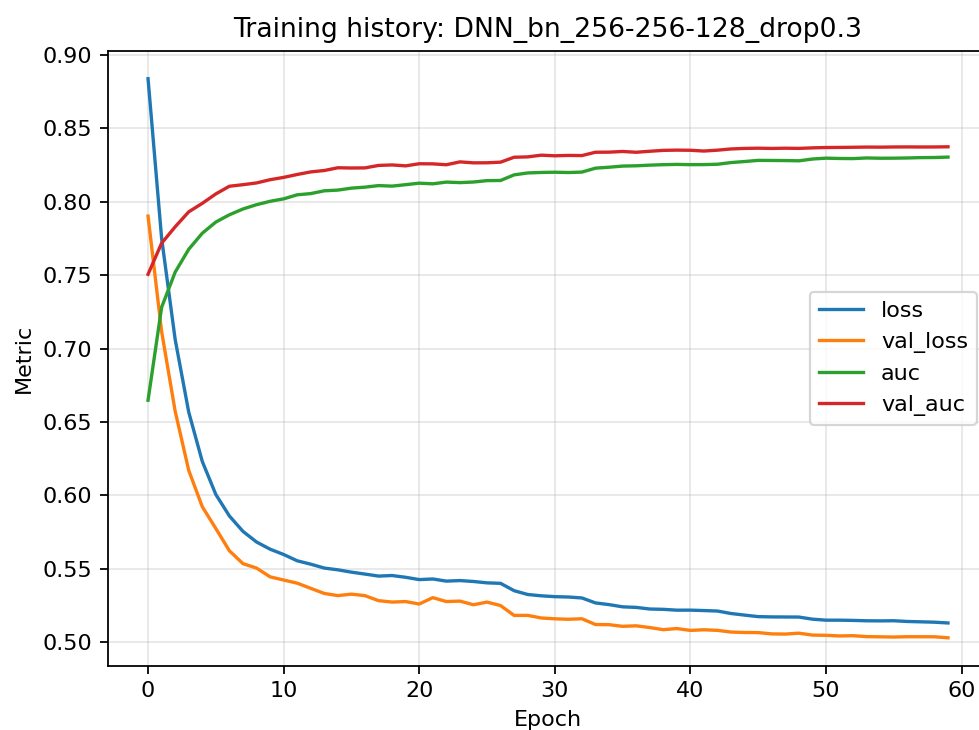


Figure 3: Zgodovina učenja za DNN z arhitekturo (256,256,128) in dropout 0.3. Tudi tukaj sta učna in validacijska krivulja blizu, kar pomeni dobro posploševanje.

4.4 Vpliv števila vhodnih spremenljivk

Nato sem preveril, kaj se zgodi, če uporabim samo podmnožico od 18 vhodnih spremenljivk. Top- k spremenljivk izberem na dva načina:

- BDT rang: spremenljivke so razvrščene po pomembnosti iz CatBoost,
- MI rang: spremenljivke so razvrščene po mutual informaciji z oznako razreda.

Za vsak k ponovno naučim model samo na teh k spremenljivkah in izmerim AUC na neodvisni validacijski množici. Za hitrost so modeli v tem skenu nekoliko enostavnejši od najboljšega modela, zato sklepam da so absolutne AUC vrednosti lahko malo nižje.

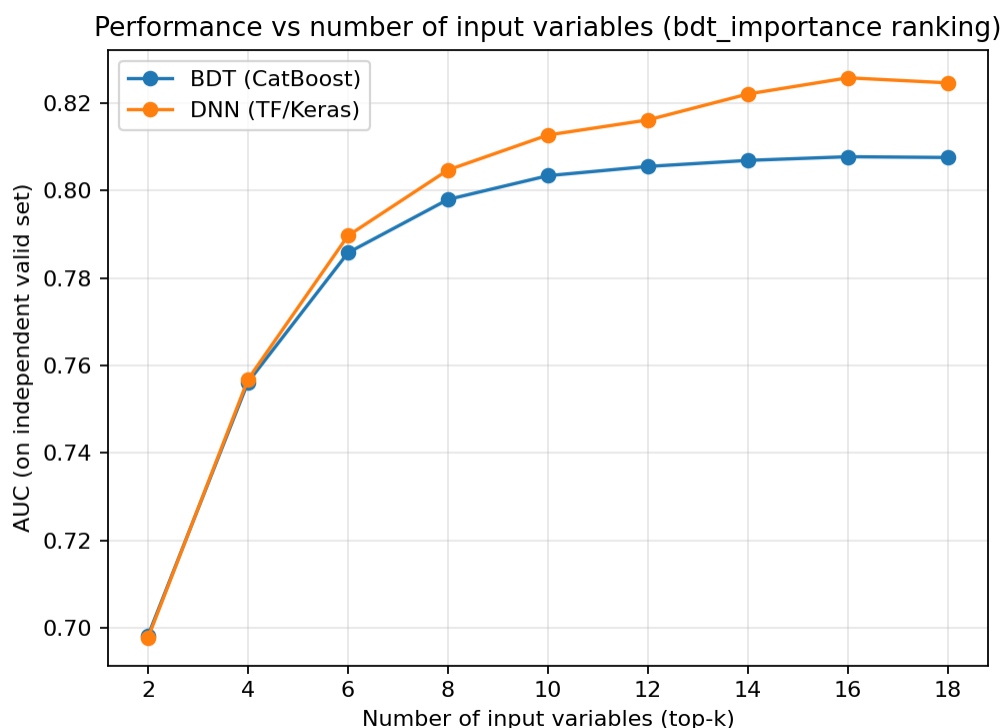


Figure 4: AUC kot funkcija števila vhodnih spremenljivk k pri razvrstitvi po BDT pomembnosti. Uspešnost močno naraste med $k = 2$ in približno $k = 10$, nato pa se izboljšave zmanjšajo.

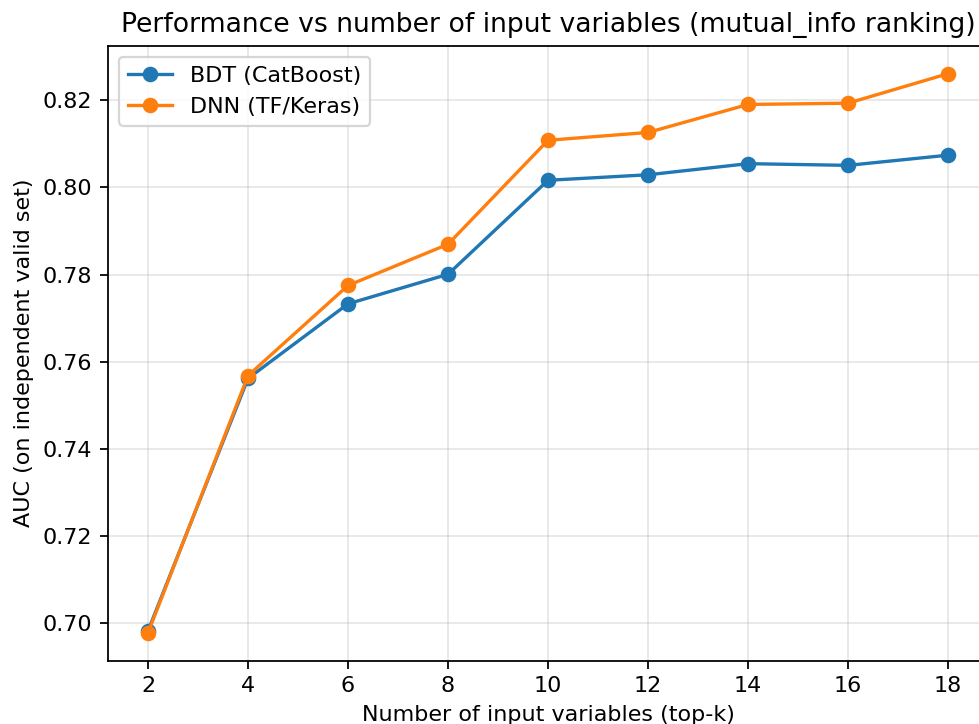


Figure 5: AUC kot funkcija k pri razvrstitvi po medsebojni informaciji. Opazimo podoben trend: velik napredek pri majhnem k in nato plato pri večjem k .

k	AUC_{BDT} (BDT rang)	AUC_{DNN} (BDT rang)	AUC_{BDT} (MI rang)	AUC_{DNN} (MI rang)
2	0.69832	0.69785	0.69832	0.69786
4	0.75620	0.75686	0.75620	0.75686
6	0.78577	0.78966	0.77332	0.77753
8	0.79793	0.80466	0.78010	0.78699
10	0.80336	0.81265	0.80165	0.81083
12	0.80547	0.81609	0.80292	0.81262
14	0.80685	0.82202	0.80546	0.81904
16	0.80769	0.82571	0.80508	0.81934
18	0.80751	0.82455	0.80741	0.82610

Table 2: AUC na neodvisni validacijski množici v odvisnosti od števila uporabljenih vhodnih spremenljivk k . Spremenljivke so dodajane glede na CatBoost pomembnost (BDT rang) ali medsebojno informacijo (MI rang).

Iz Tabele 2 in grafov 4–5 sledi:

- Zelo majhno število spremenljivk ($k = 2$ ali 4) ni dovolj in daje nizko AUC.
- Največji napredek se zgodi do približno $k \approx 10$.
- Po $k \approx 12$ – 14 se izboljšave zmanjšajo.
- DNN ima nekoliko več koristi od uporabe več spremenljivk; BDT tipično saturira nekoliko prej.

4.5 Preizkus hiperparametrov

Na koncu sem preizkusil še več nastavitev hiperparametrov. Pri BDT se AUC izboljša z več iteracijami in večjo globino dreves. Pri DNN se AUC izboljša pri večjih mrežah z dropoutom in batch normalizacijo.

Konfiguracija	iteracije	globina	η	AUC (valid)
bdt_i1000_d8_lr0.03	1000	8	0.03	0.81929
bdt_i800_d6_lr0.04	800	6	0.04	0.81374
bdt_i400_d6_lr0.06	400	6	0.06	0.80998
bdt_i200_d4_lr0.08	200	4	0.08	0.79527

Table 3: Majhen preizkus hiperparametrov za CatBoost BDT. Tukaj je η learning rate.

Konfiguracija	Skrite plasti	dropout	lr	AUC (valid)
dnn_256-256-128_drop0.3	(256,256,128)	0.3	0.0006	0.83877
dnn_128-128-64_drop0.2	(128,128,64)	0.2	0.0008	0.83433
dnn_128-64_drop0.3	(128,64)	0.3	0.0008	0.82720
dnn_64-64_drop0.2	(64,64)	0.2	0.0010	0.82031

Table 4: Majhen preizkus hiperparametrov za DNN. Dropout je uporabljen za vsako skrito plastjo.

Skupno najboljši rezultat v mojih testih doseže DNN z batch normalizacijo in dropoutom, z AUC približno 0.84 na neodvisni validacijski množici, medtem ko najboljša testirana BDT konfiguracija doseže AUC približno 0.82.

5 Zaključek

Zaključna naloga je bila kar precej zahtevna (tako zame, kot za računalnik), ampak je bilo zelo primerno, da dobro razumemo ML v časih, ko je to zelo aktualno. Sicer se mi zdi, da večina znanstvenih člankov, ki sem jih prebral o ML niso vsebovali veliko grafov, ampak so vsebovali precej več tabel. Zato sem tudi vključil toliko več tabel rezultatov v to poročilo.

Ker sem vse naloge pri tem predmetu (in tudi predtem na modelski analizi) zaganjal na službenem računalniku (in ne na lastnem, ker nisem hotel svojega porabiti) se je dobri stari HP laptop dajmo reči dodobra izrabil, tako da najbolj pozitivna stvar je, da sem v trenutku pisanja v postopku pridobivanja novega računalnika v službi :)

Sem pa hvaležen, da sem se lahko udeležil in opravljal ta predmet, saj kot meteorolog veliko raje programiram, kot pa da bi moral za izbirni predmet vzeti neke predmete (kakšne delčke ali pa astronomijo), ki niso niti približno povezani z mojim študijem in bi jih bila muka opravlјati.